OOP - THEORETICAL REF. SHEET

ASSOCIATION: Person —0..1— Car

COMPOSITION: Car ◆— Engine, Car ◆— Battery

AGGREGATION (repo-coat): Faculty ◇— 0..* Student, Faculty ◇— 1..* Teacher

DEPENDENCY: Car ⇢ Gasoline

INHERITANCE: Animal ← Dog, Animal ← Penguin

pointer / reference + virtual => REAL obj. type.

pointer / reference + non-virtual => POINTER TYPE.

ex: `B* b[] = {new B{}, new D{}};`

- b[0] = B* => pointer type = B*
  - ↓ new B{} => real obj. type = B
- b[1] = B* => pointer type = B*
  - ↓ new D{} => real obj. type = D

b[0] -> print()
↓
NON-VIRTUAL
⇓
pointer type => B*
=> B::print()

b[1] -> print()
↓
NON-VIRTUAL
↓
pointer type => B*
=> B::print()

- beverage => private member
- 0..* => zero or more (aggregation)
- 1..* => one or more (aggregation)
- 0..1 => zero or one (association)

b[0] -> print()
↓
#1 => one-to one (icecream)
1 => one-to-one (one encoder)
2 =>
↓
VIRTUAL ~~in base class~~
⇓
REAL obj. type => B => B::print()

b[1] -> print()
↓
Still VIRTUAL
⇓
REAL obj. type => D => D::print()

B b -> new object
B& b -> alias, no obj.
B* b -> alias, no obj.

catch-by-object
↳ born at throw, dies inside catch after data is copied

catch-by-value
↳ born at catch, dies at catch }

dequeue = remove + return elem. from the front

bool operator==(const Complex& other) const {
return this->real == other.real &&
this->imaginary == other.imaginary

ITERATOR: itBegin = d.begin() => points to FIRST elem.
itEnd = d.end() => points ~~AFTER~~ the LAST elem.

Rule of three: -> if ∄ one of these => when it goes out of scope => deletes twice => dangling pointer (if ∄ copy constructor / ∄ assignment operator)
1) copy constructor
2) destructor
3) assignment operator

Shallow copy - copies the pointer
Deep copy - allocates new memory

-x => parameter
x = attribute (distinction)

```
vector v2 = v1 //no default copy constructor => shallow copy.
a3 = a1; => shallow copy
A a4 = a3 => shallow copy
```

By overriding a virtual fct, the child class completely replaces the base class behaviour.

If in base class the fct f prints "a" and it's virtual in child class the overriden fct f that prints "b" will not print ab, only b.

Base::f before print b if you want "ab"

OBSERVATIONS:
- we can't create obj. from abstract classes!
- nothing after throw() gets executed!
  - => directly in catch (if finds cond.)
  - => if not caught => error.
- if an object is thrown, it gets destroyed after catch.
- catch (Ex1 e) => catch BY VALUE => copy constructor.
- child class function f() is virtual as well if the parent class function f() is virtual, (even if in the child class is not marked as virtual).
- d -> g(*b) => copy constructor...
- OUT OF SCOPE! => destructor automatically called!
- Beverage* beverage => beverage -> print() (pointer) -> no constr.
- Beverage beverage => beverage.print() (object) -> constr.

v.insert(it, x) => inserts x BEFORE it.
*it = 11 => change VALUE.

```
str.erase(it);
str.insert(it, "b");
```
} ERROR (uses invalid it.)

```
it = str.erase(it);
str.insert(it, "b");
```
} CORRECT

end() - 1 = LAST ELEMENT

pop_back() -> delete LAST element.

```
v.erase(it);
it = v.begin() + 3;
```
} CORRECT (Iterator reset)

front_inserter(...) => inverse order.
back_inserter() => normal order.

double delete (dangling ptr)

a3 = a1 => automatic assignment op.

A a4 = a1 => copy ctr

If ∄ copy ctr, ∄ assignment op, ∄ destructor => compiler generates automatically
⇓
SHALLOW

- When we have a vector of pointers, destructor is called automatically when main() goes out of scope; we need to do delete d only when we have sth. like: `SumDiscount* d = new SumDiscount{};`
- As long as the inheriting (child) class does not define pointers, it doesn't need to override the parents destructor;

Encoder <-2- Mixer
=> inside Mixer class: Encoder* e1, Encoder* e2

Sale ◇— 0..* SaleItem => Sale contains (0 or several SaleItem more)
=> inside Sale class: std::vector<SaleItem*> items.

SumDiscount ◇— 1..* Discount => SumDiscount contains several (1 or more) Discounts
=> inside SumDiscount class: std::vector<Discount*> discounts;

MenuItem — 0..1 action — Action
=> inside MenuItem class: Action* action; MenuItem(..., Action* action = nullptr) : ...

#1
Icecream
IcecreamWithTopping
=> inside IcecreamWithTopping class: Icecream* icecream;

Expression
#right
#left
Negate — UnaryExpression — BinaryExpression
=> inside UnaryExpr: Expression*
=> inside BinaryExpr: Expression*
right; + inheritance
left + inheritance from UnaryExpr.

```
bool operator==(const Complex& other) const{
    return this->real==other.real &&
           this->imaginary==other.imaginary;
}
```

```
Complex operator/(int value) const{
    if(value==0) throw...
    return Complex{this->real/value,...};
}
```

```
friend ostream& operator<<(ostream& out, const Complex& c)
ostream& operator<<(ostream& out, const Complex& c){
    out<<c.real<<"+"<<c.imaginary<<"i";
    return out;
}
```

```
HTMLBuilder& operator+=(T elem){
    if(..) throw...
    elems.push_back(elem);
    return *this;
}
```

```
Vector<T>& add(const T& elem){
    elems.push_back(elem);
    return *this;
}
```

```
Adder& operator-
    if(values.size()==
        throw.
    values.pop_back(.
    return *this;
}
```

← CHAINING (.add.add..

```
Adder& operator++(){
    values.push_back(values.back(
    return this;
}
```

← inside the class

```
Adder& operator+(T value){
    values.push_back(value);
    return *this;
}
```

```
T& operator*() const{ return *ptr};
```

Search dynamically:

```
ui->listWidget->clear();
std::string text=ui->lineEdit->text().toStdString();
if(text.empty())
    return;
auto question=service.get...(text);
ui->listWidget->addItem(QString....);
auto answers=service.get...(question);
for(const auto& a:answers){
    ui->listWidget->addItem(QString::fromStdString("\t"+a.toString()));
}
```

← tabs indentation.

```
Question Service::getBestMatching(...text){
    auto questions=repo.getQuestions;
    std::sort(q...begin(),....end(),[this,text]
    (const Question& q1,...q2){
        return similarityScore(q1.getText(),text)>
        sim...(q2...,text);
    });
} return questions[0];
```

connect(ui->lineEdit,&QLineEdit::textChanged,this,...);

```
T sum() const{
    T s=values[0];
    for(int i=1; i<values.size(); i++)
        s=s+values[i];
    return s;
}
```

```
template<typename T>
class Vector{
    private:
        vector<T> elems;
    public
        ...
}
```

Clarify Handrail{...}, vs handrail{handrail} etc.

I. child class needs all parent's fields!

(H1)

```
class Parent{
    private:
        int x; int y;
    public:
        Parent(int x,int y):x{x},y{y}{}
};
class Child:public Parent{
    public:
        Child(int x,int y):Parent{x,y}{}
};
```

can be done only when we inherit

(H2)

```
class Child:public Parent{
    private:
        int x,int y;
    public:
        Child(int x,int y):
        :x{x},y{y}{}
};
```

NEVER!

```
Child(int x,int y){
} this->x=x;
```

~~copy constructor~~

~~unique_ptr<T>&&~~

II. parent fields private => provide getters/setters; protected => no getters/setters

III. child passes extra fields on its constructor: Child(int x, int y, int extra):Parent{x,y},extra{extra}{}

IV. Dependency injection: class PinkBag:public Bag{
    private: GoldChain* chain;
    public:
        PinkBag(....,GoldChain* chain):....,chain{chain}{}

V. Internal composition:
class Beverage(...):descrip{descrip}{}
...
BeverageWithMilk(Beverage* beverage, int cnt):Beverage{"w milk"},beverage{beverage},cnt{cnt}{}
= nullptr => optional param → at the END (last field of the constructor!)

Patterns

```
template<typename T>
class ... {
  private:
    std::vector<T> elems;
};
```

when you see for(auto x : todo) → you need functions for begin() and end()! auto begin(){return elems.begin();}

```
auto begin(){ return elems.begin(); };
auto end(){ return elems.end(); }
```

operator+

```
Adder& operator+(T value){
    values.push_back(value);
    return *this;
}
```

```
Set<T> operator+(const T& elem) const{
  Set<T> result = *this;
  for... throw...
  result.elems.push_back(elem);
  return result;
}
```

```
Stack<T> operator+(const T& elem) const{
  if .... throw...
  Stack<T> result = *this;
  result.elems.push_back(elem);
  return result;
}
```

operator++

```
Adder& operator++(){
  values.push_back(values.back());
  return *this;
}
```

```
friend std::ostream& operator<<(std::ostream& out, const Act& a)
{ out << "Activity" << a.name << " at " << a.hour;
  return out;
}
```

operator*

```
T& operator*() const{
    return *ptr;
}
```

```
char operator*() const{
    return value;
}
```

operator-

```
Vector<T> operator-(const T& elem){
  Vector<T> result;
  bool found = false;
  for(...){
    if(!found && current == elem)
        found = true;
    else
        result.add(current);
  }
}
```

operator--(')

```
Adder& operator--(){
  if(values.size() == 1)
      throw....
  values.pop_back();
  return *this;
}
```

```
bool operator+=(
const T& e){
  elems.push_back(e);
  return *this;
}
```

operator[]

```
T operator[](int i){
  return elems[i];
}
```

```
template<typename ID,
typename T>
class Repo{
  private:
    std::vector<std::pair<
      ID, T>> elems
};
```

operator==

```
bool operator==(const Complex& other) const{
  return this->real == other.real &&
         this->imag == other.imag;
}
```

```
bool operator==(const SmartPointer<T>& other){
    return *ptr == *other.ptr;
}
```

```
MyObjects& addBeginning(Object * object){
  objects.insert(objects.begin(), object);
  return *this;
}
```

operator+=

```
HTMLBuilder& operator+=(T elem){
  if(elem == nullptr) throw...
  elems.push_back(elem);
  return *this;
}
```

```
ToDo<T>& operator+=(const T& elem){
  elems.push_back(elem);
  return *this;
}
```

operator=

```
SmartP<T>& operator=(const SmartP<T>& other)
{ if(this == &other) return *this;
  (*count)--;
  if(*count == 0){ delete ptr; delete count; }
  ptr = other.ptr;
  count = other.count;
  (*count)++;
  return *this;
}
```

```
void add(ID id, const
T& obj){
  elems.push_back({id, obj});
}
```

operator/

```
Complex operator/(int value) const{
  if(value == 0) throw....
  return Complex{this->real/value,
                 this->imag/value};
```

sum

```
T sum() const{
  T s = values[0];
  for(int i=1; i<values.size(); i++)
      s = s + values[i];
  return s;
}
```

max

```
T fct(const vector<T>&v)
{ T max = v[0];
  for(const T& elem : v){
    if(elem > max)
        max = elem;
  }
  return max
```

operator<<

inside class:

```
friend ostream& operator<<(ostream& out,
                           const Complex& c)
```

outside class:

```
ostream& operator<<(ostream& out,
                    const Complex& c){
  out << c.real << "+" << c.imag << "i";
  return out;
}
```

reverse print:

```
for(auto it = elems.rbegin();
it != elems.rend(); ++it) { out << *it << "\n"; }
```

pop (stack)

```
T pop(){
  T e = elems.back();
  elems.pop_back();
  return e;
}
```

pop (queue)

```
T pop(){
  T e = elems.front();
  elems.erase(elems.
    begin());
  return e;
}
```